

A. Yes, some extensions are required, particularly in the telecom industry, where the primary focus is on passing packets quickly. Kubernetes is designed to run in environments where nodes are elastically scalable and generic—but that’s not always the case in telecom. So we extend the Kubernetes API to create custom resources using custom resource definitions (CRDs) to manage the infrastructure. When sending an upgrade plan to the downstream cluster, these CRDs coordinate the rolling window upgrade for each individual node in the cluster.

Q. How does SUSE equip the user to handle the deployment and management of different functionalities?

A. Our failure domain model allows each remote location to run independently without requiring permanent connectivity. In this model, the remote cluster (or downstream cluster) regularly phones home to Rancher for task updates. It pulls down a declarative manifest that specifies the necessary changes, and Kubernetes then ensures the system aligns with the declared state. If Rancher doesn’t have any tasks, the cluster continues operating locally.

Q. What do you mean by open standards, and how are these different from open source?

A. The difference between open standards and open source lies in their purpose. Open source is about the collaborative development of software, while open standards refer to the agreed-upon protocols, APIs, or interfaces that emerge from these collaborations. Open standards ensure uniformity and interoperability, allowing anyone to build on top of them, benefiting everyone in the ecosystem. Open source fosters collaboration, and the result is often the creation of open standards that facilitate innovation and widespread adoption.

Q. Could you give us examples of open source projects that have led to the adoption of Kubernetes as a standard?

A. The Sylva Project—a Linux Foundation open source community, of which SUSE is a part—aims to address 5G challenges, and they’ve chosen cloud-native technologies to do so. Kubernetes was selected as the infrastructure platform for use at the tower and within the core network. Similarly, major players in the industrial IoT (IIoT) space have come together within the Margo (at margo.org) initiative, by independently selecting Kubernetes as the standard API for industrial process automation services. These communities embrace the spirit of open source and collaborate to establish standardised specifications and implementations. This allows anyone globally to build value on top of these standards or APIs.

Q. How does selecting Kubernetes as an open standard impact interoperability across different kinds of edge hardware or platforms?

A. Kubernetes functions as an infrastructure platform that abstracts the underlying hardware. Its biggest value lies in its declarative API, which allows you to define your infrastructure needs, simplifying management by ensuring the system aligns with your configuration without manual intervention.

Beyond the basic layer, in the IIoT space, there are various types of devices—sensors, actuators, IP cameras, etc—that are too small to run Linux or Kubernetes but still operate using different protocols. Some vendors even add proprietary modifications, which makes interoperability across these devices challenging, especially since many of the protocols are industry specific. This is where open standards come into play.

The Margo project and Akri, an initiative started by Microsoft, address

these interoperability challenges. Akri standardised the discovery of IIoT devices and unifies the way Kubernetes communicates with them. The Margo project complements this by focusing on firmware management, security, and process automation for these devices. Margo helps with managing credentials and ensuring IIoT devices receive firmware updates. Together, Akri and Margo create a standardisation framework that improves interoperability and reduces fragmentation across diverse edge hardware and platforms.

Q. How can Kubernetes handle any connectivity or integration challenge that arises in hybrid cloud connections?

A. Each on-premises location (on edge) functions as a downstream cluster with its own Kubernetes API. On the cloud side, a cloud-hosted Kubernetes cluster is deployed, providing the same API. In a hybrid environment, tools such as Rancher act as a multi-cluster management tool that bridges the gap between on-premises (edge) clusters and cloud clusters. Rancher allows you to have a control plane in the cloud that manages all the downstream clusters, whether they’re in the cloud or onsite, all using the same Kubernetes API. It provides the connectivity and visibility needed to manage both environments as a unified system, solving the challenges of hybrid cloud connections effectively.

Q. How do you manage scalability across multiple platforms at the edge?

A. Scalability challenges at the edge can be addressed by creating small, independent failure domains and Kubernetes’ declarative infrastructure. Automated testing pipelines ensure that once a declaration is validated, it can be replicated across multiple locations without human intervention. This approach avoids errors and ensures that scaling can be handled effectively.